



UNCUYO
UNIVERSIDAD
NACIONAL DE CUYO

PARADIGMAS DE PROGRAMACIÓN

UNIDAD 5

Dr. Pablo Javier Vidal
Ing. Laura Noussan Lettry

Facultad de Ingeniería
Universidad Nacional de Cuyo

2026

- 1 Gestión en C
- 2 Memoria libre en lenguajes dinámicos
- 3 Garbage collector
- 4 Contador de referencias

Objetivos

- Entender qué significa **memoria libre** y qué significa **memoria no referenciada**.
- Relacionar **stack**, **heap**, referencias y objetos durante la ejecución.
- Comparar **administración manual** y **garbage collection**.
- Presentar dos estrategias clásicas: **contadores de referencias** y **tracing GC**.
- Ver un **ejemplo en Java** y una comparación breve con **Python** y **C**.
- Explicar cómo se coordinan el **runtime del lenguaje** y el **Sistema Operativo**.

Idea central

Pregunta guía

¿Cuándo un bloque del heap puede volver a usarse?

- En **C**, el programador decide con **malloc/free**.
- En lenguajes con **runtime**, el sistema del lenguaje intenta detectar qué objetos **ya no son alcanzables**.
- La clave no es si el objeto *existe*, sino si todavía puede ser **alcanzado por alguna referencia activa**.

Idea para recordar

Memoria libre = espacio reutilizable.

Memoria no referenciada = objeto que ya no puede ser alcanzado desde el programa activo.

Base: administración manual en C

```
#include <stdlib.h>

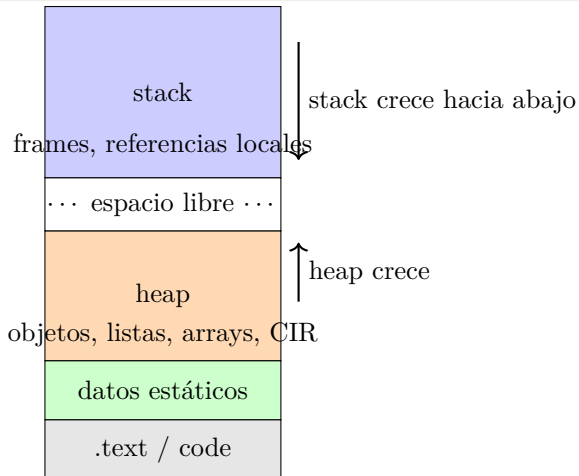
typedef struct Nodo {
    int valor;
    struct Nodo *sig;
} Nodo;

int main(void) {
    Nodo *p = malloc(sizeof(Nodo));
    p->valor = 10;
    p->sig = NULL;

    /* usar p */

    free(p);
    p = NULL;
    return 0;
}
```

Mapa conceptual de memoria



Lectura

El **stack** conserva el contexto activo de ejecución

Gestión en C

Lenguajes sin Garbage Collector

En algunos lenguajes, la memoria no se libera automáticamente.

Idea clave

El programador es responsable de:

- reservar memoria
 - utilizarla correctamente
 - liberarla cuando ya no se usa
-
- No hay recolección automática.
 - No existe análisis de alcanzabilidad.
 - No hay chequeo de referencias vivas.

Ejemplo en C: manejo manual

```
#include <stdlib.h>

int main() {
    int *p = malloc(sizeof(int));

    *p = 10;

    free(p);

    return 0;
}
```

- malloc reserva memoria en el heap.
- free libera la memoria.

Problemas del manejo manual

- **Memory leak:**
 - memoria nunca liberada
- **Dangling pointer:**
 - puntero a memoria liberada
- **Double free:**
 - liberar dos veces la misma memoria
- **Uso después de liberar:**
 - comportamiento indefinido

Ejemplo de error (memory leak)

```
int *crear() {  
    int *p = malloc(sizeof(int));  
    *p = 5;  
    return p;  
}
```

```
int main() {  
    int *x = crear();  
    // falta free(x)  
    return 0;  
}
```

Problema

La memoria queda reservada pero nunca se libera.

Ejemplo de error (dangling pointer)

```
int *p = malloc(sizeof(int));  
  
free(p);  
  
*p = 10; // ERROR
```

Problema

Se accede a memoria ya liberada.

Modelo de memoria en C



- El heap es gestionado manualmente.
- El sistema operativo solo entrega memoria.
- El lenguaje no sabe cuándo liberar.

¿Qué hace el Sistema Operativo?

- Provee memoria al proceso (heap).
- No sabe qué datos están en uso.
- No puede liberar memoria automáticamente.

Idea clave

El SO gestiona memoria a nivel de proceso, no a nivel de objetos del programa.

Lenguajes sin Garbage Collector

Lenguajes que manejan memoria manualmente:

- C
- C++
- Rust (sin GC, pero con ownership)
- Assembly

Diferencias

- C / C++: manejo manual explícito
- Rust: control en compilación (ownership)

¿Qué es el Ownership?

Es el sistema de Rust para gestionar la memoria mediante un conjunto de reglas que el compilador verifica en tiempo de compilación.

Las Tres Reglas de Oro

- 1 Cada valor en Rust tiene una variable que es su **dueño** (owner).
- 2 Solo puede haber **un dueño a la vez**.
- 3 Cuando el dueño sale de su **ámbito** (scope), el valor se descarta automáticamente.

Ejemplo de código: El chequeo del compilador

El *Borrow Checker* impide que usemos una variable que ha perdido su propiedad.

```
fn main() {  
    let s1 = String::from("Hola");  
    let s2 = s1; // Propiedad movida a s2  
  
    // println!("{}", s1);  
    // ^--- ERROR: value borrowed here after move  
  
    println!("{}", s2); // Esto funciona  
}
```

- **Copia (Copy):** Los tipos simples (integers, bools) residen en el stack y se copian automáticamente sin perder propiedad.

Ventajas del Ownership

- **Seguridad de memoria:** Evita punteros colgantes (*dangling pointers*).
- **Sin Garbage Collector:** No hay pausas en tiempo de ejecución para limpiar memoria.
- **Eficiencia:** Gestión determinista de recursos (RAII).
- **Concurrencia segura:** Evita condiciones de carrera (*data races*) al asegurar acceso exclusivo.

Desventajas y Desafíos

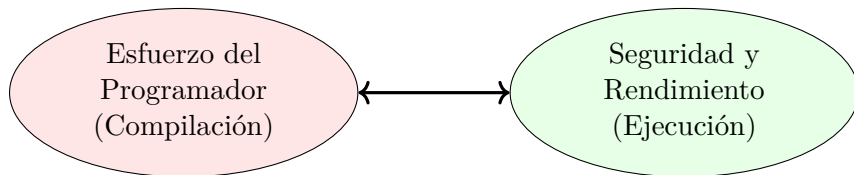
Curva de Aprendizaje

Es el concepto más difícil para los programadores que vienen de lenguajes con GC (Python, Java) o gestión manual (C).

- **Rigidez:** Ciertas estructuras de datos (como grafos cíclicos o listas doblemente enlazadas) son difíciles de implementar con propiedad simple.
- **Lucha con el Borrow Checker:** El compilador puede ser frustrante al principio al rechazar código que "parece correcto".
- **Complejidad en APIs:** Obliga a pensar en el tiempo de vida de los datos (*lifetimes*) al diseñar funciones.

Conclusión

El **Ownership** es un compromiso:



Cambiamos esfuerzo mental durante el desarrollo por programas extremadamente rápidos y libres de errores de memoria en producción.

Comparación admin. manual con Garbage Collector

Característica	C	Java/Python
Liberación	manual	automática
Errores	frecuentes	menos frecuentes
Control	alto	menor
Performance	predecible	depende del GC

Idea final

Lenguajes sin GC

El programador decide cuándo liberar memoria.

Lenguajes con GC

El runtime decide cuándo liberar memoria.

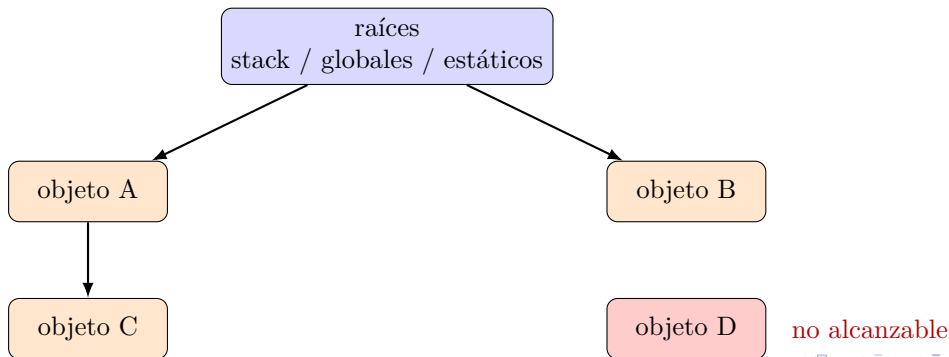
Trade-off

Control vs Seguridad

Memoria libre en lenguajes dinámicos

Memoria libre en lenguajes dinámicos

- En Java, Python y otros lenguajes administrados, los objetos viven en un **heap manejado por el runtime**.
- El programador crea refs. y objetos, pero la liberación suele ser **automática**.
- El GC razona sobre el **grafo de objetos alcanzables**, no sobre una variable aislada.



¿Cómo se identifican fragmentos no referenciados?

Criterio general

Un objeto es **basura** si no existe un camino desde las **raíces** del programa hasta ese objeto.

Raíces típicas

- referencias en el **stack**
- variables **globales** o **static**
- registros de máquina usados por el runtime
- estructuras internas del intérprete o VM

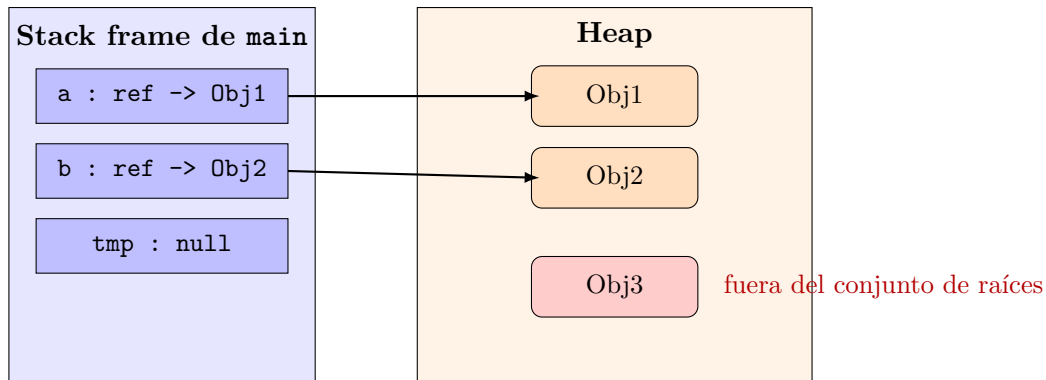
Conclusión

- si un objeto es alcanzable, se conserva
- si es inalcanzable, puede reciclarse
- el GC recupera **bloques aislados** y también **subgrafos enteros**

No alcanza con mirar una variable

Salir de scope no implica recolección inmediata; cambia la **alcanzabilidad** del

La pila y las raíces del recolector

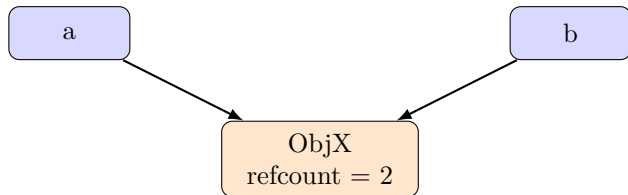


Mensaje clave

El recolector parte de las referencias activas en la **pila** y desde allí explora el heap.

Contadores de referencias

- Cada objeto mantiene un **contador** con la cantidad de referencias que apuntan a él.
- Cuando una nueva referencia aparece, el contador **aumenta**.
- Cuando una referencia desaparece o cambia, el contador **disminuye**.
- Si el contador llega a **0**, el objeto puede liberarse.



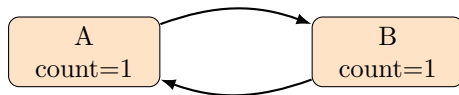
Ventajas y problema de los contadores

Ventajas

- modelo intuitivo
- liberación rápida de objetos que quedan sin referencias
- útil para administrar recursos en tiempo cercano al descarte

Problema clásico: ciclos

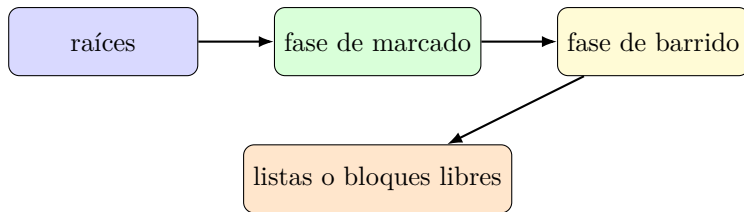
Dos objetos pueden referenciarse entre sí y, aun así, quedar **inalcanzables desde el programa**.



ninguna raíz del programa llega a A o B

Garbage collector por trazado (mark and sweep)

- 1 Se parte del conjunto de **raíces**.
- 2 Se recorren referencias y se **marcan** los objetos alcanzables.
- 3 Luego se barre el heap y se liberan los no marcados.



Consecuencia

A diferencia del conteo de referencias, esta estrategia puede detectar **ciclos inalcanzables**.

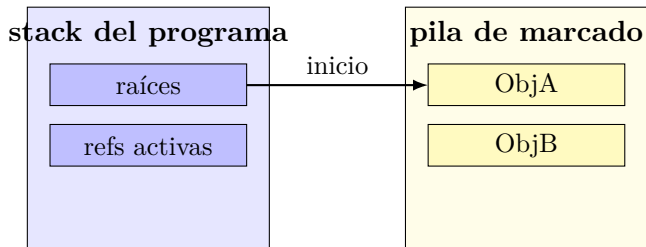
La pila del GC: raíces y pila de marcado

1. Pila del programa

- contiene variables locales activas
- funciona como **conjunto de raíces**
- el GC la inspecciona para iniciar el recorrido

2. Pila de marcado del recolector

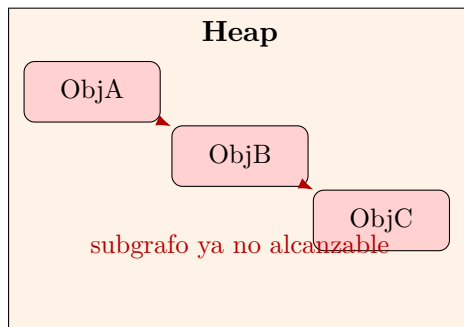
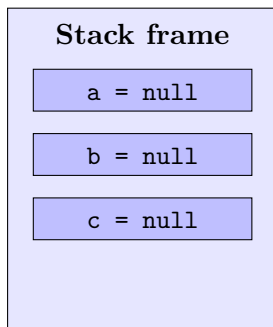
- estructura interna del algoritmo
- guarda objetos pendientes de explorar
- evita perder nodos del grafo durante el marcado



Ejemplo en Java

```
class Nodo {  
    String nombre;  
    Nodo sig;  
    Nodo(String n) { nombre = n; }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Nodo a = new Nodo("A");  
        Nodo b = new Nodo("B");  
        Nodo c = new Nodo("C");  
        a.sig = b;  
        b.sig = c;  
        b = null;  
        a = null;  
        c = null;  
    }  
}
```

Ejemplo en Java: lectura paso a paso



- Al poner `a = null`, `b = null` y `c = null`, se pierde el acceso al subgrafo.
- A, B y C quedan inalcanzables.
- El GC puede recuperar **los tres objetos** aunque aún se referencien entre sí dentro del heap.

Python y Java: misma idea, distinta implementación

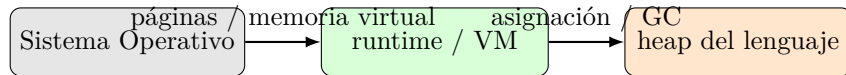
Lenguaje	Idea dominante	Comentario didáctico
C	liberación manual	el programador controla <code>malloc/free</code> ; hay riesgo de fuga, doble liberación y uso luego de liberar
Java	GC por trazado generacional, sobre heap administrado	el runtime decide cuándo recolectar; la alcanzabilidad manda
Python	conteo de referencias + recolector de ciclos	libera muchos objetos de inmediato por refcount, y usa GC adicional para ciclos

Punto en común

En todos los casos hay heap, referencias y objetos. Lo que cambia es **quién decide** cuándo un bloque vuelve a ser memoria libre.

Coordinación entre el Sistema Operativo y el lenguaje

- El **Sistema Operativo** entrega memoria al proceso: páginas, segmentos, espacio virtual.
- El **runtime del lenguaje** administra ese espacio para crear y reciclar objetos del heap.
- El GC no suele devolver cada objeto directamente al SO; primero lo reutiliza dentro del **heap administrado**.



Lectura

El SO administra memoria del **proceso**. El lenguaje administra memoria de **objetos** dentro de ese proceso.

Esquema completo del proceso

- 1 El programa crea objetos en el **heap**.
- 2 Las referencias activas viven en **stack**, globales o estructuras del runtime.
- 3 El runtime detecta presión de memoria o alcanza un umbral.
- 4 El GC parte de las **raíces** y determina qué objetos siguen vivos.
- 5 Los objetos inalcanzables se convierten en **memoria libre reutilizable**.
- 6 Eventualmente, el runtime puede pedir más memoria al SO o devolver parte del heap si su estrategia lo permite.

Idea final

La recolección de basura es un mecanismo para **reconstruir automáticamente la frontera entre memoria viva y memoria libre**.

Resumen

- **Memoria libre** no es solo memoria vacía: es memoria que el runtime puede volver a asignar.
- La noción fundamental es la **alcanzabilidad** desde raíces activas.
- El **stack** aporta raíces; el **heap** contiene los objetos.
- **Contadores de referencias** son intuitivos, pero fallan con ciclos.
- Un **garbage collector por trazado** puede identificar subgrafos enteros inalcanzables.
- Java y Python automatizan la recolección con estrategias diferentes; C deja el control al programador.

Garbage collector

Garbage Collector: idea general

Pregunta guía

¿Cómo sabe el runtime qué objetos del heap siguen vivos y cuáles pueden liberarse?

- El **heap** contiene objetos (CIR).
- Las **raíces** del programa están en el stack, variables estáticas y referencias del runtime.
- El GC no busca objetos “útiles”: busca objetos **alcanzables**.

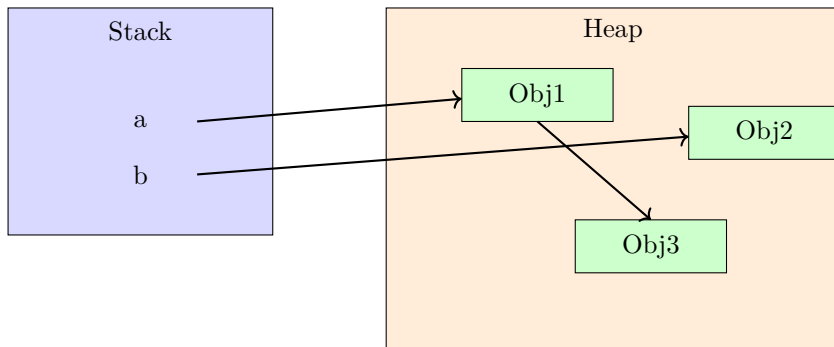
Objetivo del GC

Liberar objetos no alcanzables sin intervención del programador.

Idea clave

Un objeto vive mientras exista un camino de referencias desde alguna raíz hasta su CIR.

Raíces del Garbage Collector



Raíces

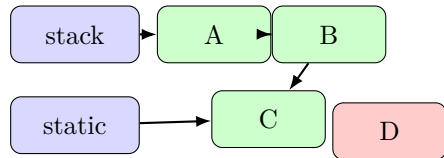
Las referencias en el stack son el punto de inicio del GC.

¿Qué es el grafo de objetos alcanzables?

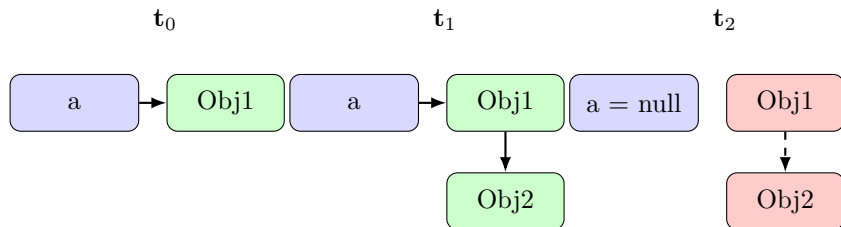
- **Nodos:** objetos del heap.
- **Aristas:** referencias entre objetos.
- **Raíces:** stack, campos estáticos, referencias JNI, etc.

Muy importante

El runtime no suele mantener un “grafo explícito” permanente. Lo **reconstruye implícitamente** recorriendo referencias reales cada vez que ejecuta una recolección.



El grafo no es fijo: cambia durante la ejecución



- En t_0 existe un objeto alcanzable.
- En t_1 el objeto referencia a otro y el grafo crece.
- En t_2 la raíz desaparece: los dos objetos quedan **inalcanzables**.

Conclusión

El grafo cambia continuamente. Por eso el GC lo vuelve a recorrer cada vez que necesita recolectar.

¿Se construye una sola vez? ¿Cada cuánto se recorre?

- **No** se construye una sola vez.
- **No** suele recorrerse “cada X segundos” de manera fija.
- El runtime lanza el GC cuando lo indican sus **heurísticas**:
 - falta de espacio en el heap,
 - llenado de una generación joven,
 - presión de memoria,
 - pedido explícito del runtime.

En Java

El heap se divide en regiones/generaciones. Las colecciones jóvenes suelen ser más frecuentes; las colecciones mayores son menos frecuentes y más costosas.

Idea clave

En cada ciclo de GC, el runtime vuelve a descubrir qué objetos son alcanzables desde las raíces actuales.

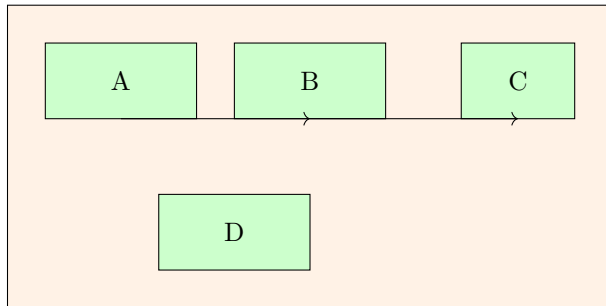
Algoritmo Mark & Sweep

- ① **Mark:** recorrer desde raíces y marcar objetos alcanzables
- ② **Sweep:** liberar objetos no marcados

Idea clave

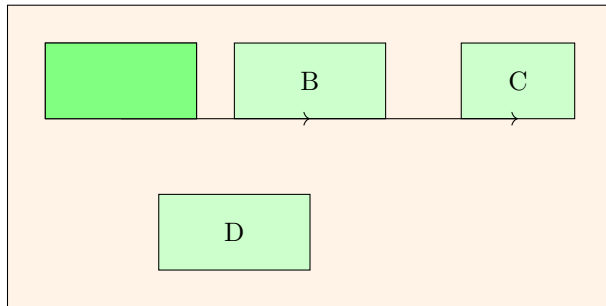
Se basa en el concepto de **alcanzabilidad**

Mark & Sweep: paso 1 (mark)



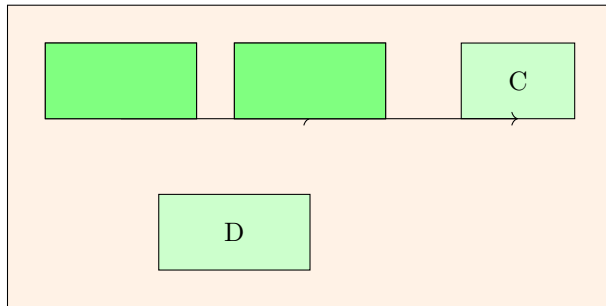
- Se marca A
- Se alcanza y marca B
- Se alcanza y marca C

Mark & Sweep: paso 1 (mark)



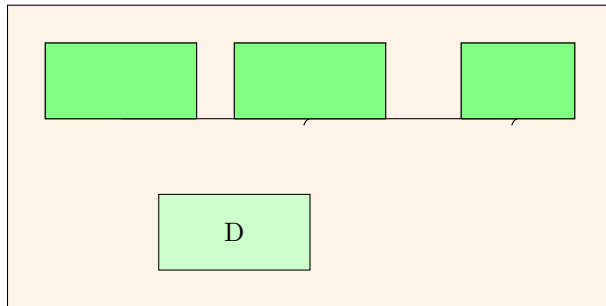
- Se marca A
- Se alcanza y marca B
- Se alcanza y marca C

Mark & Sweep: paso 1 (mark)



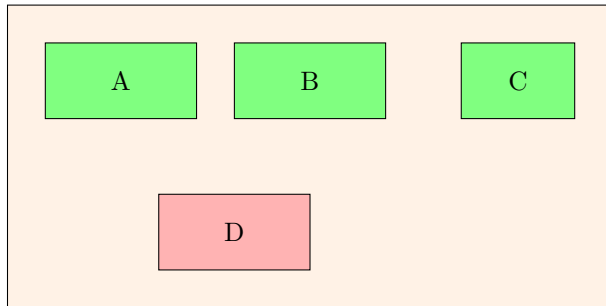
- Se marca A
- Se alcanza y marca B
- Se alcanza y marca C

Mark & Sweep: paso 1 (mark)



- Se marca A
- Se alcanza y marca B
- Se alcanza y marca C

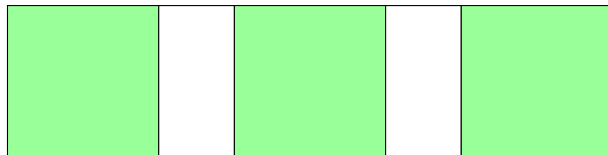
Mark & Sweep: paso 2 (sweep)



Resultado

El objeto D no fue marcado → se libera

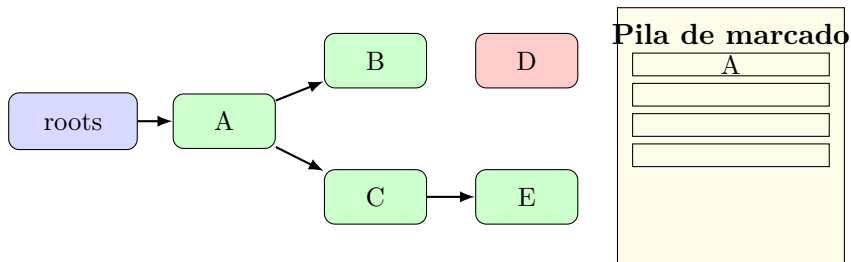
Problema: fragmentación



huecos en memoria

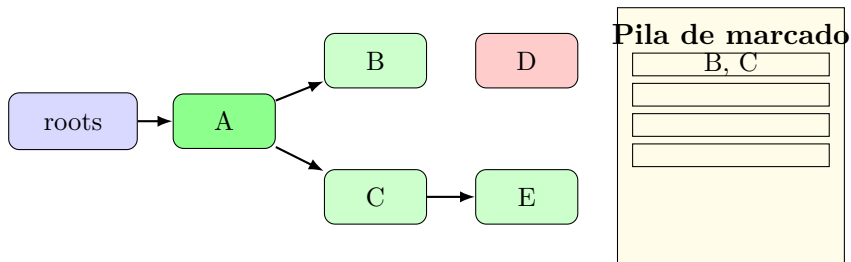
- Memoria libre no contigua
- Problemas para nuevas asignaciones

Mark & Sweep: recorrido DFS paso a paso



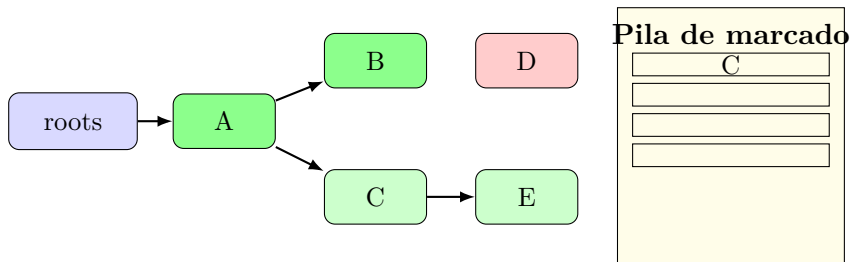
- 1. Se parte de las raíces y se marca **A**.
- 2. DFS sigue una referencia y marca **B**.
- 3. Vuelve, marca **C** y descubre **E**.
- 4. Al terminar, **D** no fue alcanzado.

Mark & Sweep: recorrido DFS paso a paso



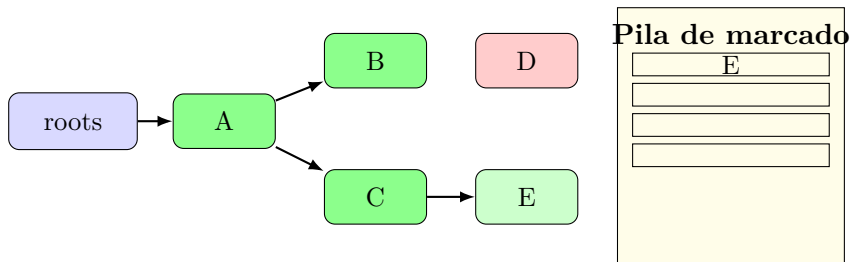
- 1. Se parte de las raíces y se marca **A**.
- 2. DFS sigue una referencia y marca **B**.
- 3. Vuelve, marca **C** y descubre **E**.
- 4. Al terminar, **D** no fue alcanzado.

Mark & Sweep: recorrido DFS paso a paso



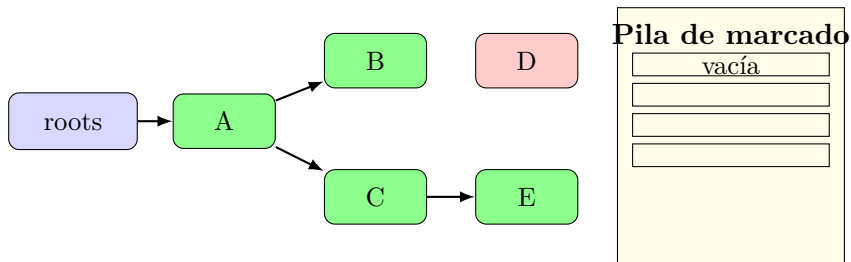
- 1. Se parte de las raíces y se marca **A**.
- 2. DFS sigue una referencia y marca **B**.
- 3. Vuelve, marca **C** y descubre **E**.
- 4. Al terminar, **D** no fue alcanzado.

Mark & Sweep: recorrido DFS paso a paso



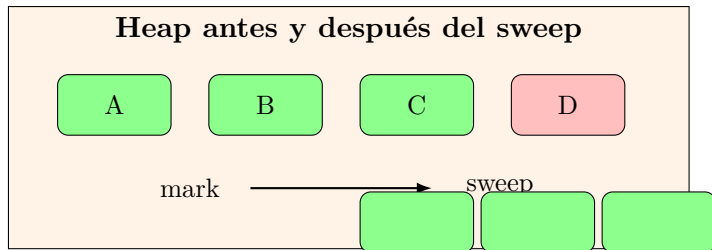
- 1. Se parte de las raíces y se marca **A**.
- 2. DFS sigue una referencia y marca **B**.
- 3. Vuelve, marca **C** y descubre **E**.
- 4. Al terminar, **D** no fue alcanzado.

Mark & Sweep: recorrido DFS paso a paso



- 1. Se parte de las raíces y se marca **A**.
- 2. DFS sigue una referencia y marca **B**.
- 3. Vuelve, marca **C** y descubre **E**.
- 4. Al terminar, **D** no fue alcanzado.

Fase sweep: liberar lo no alcanzable

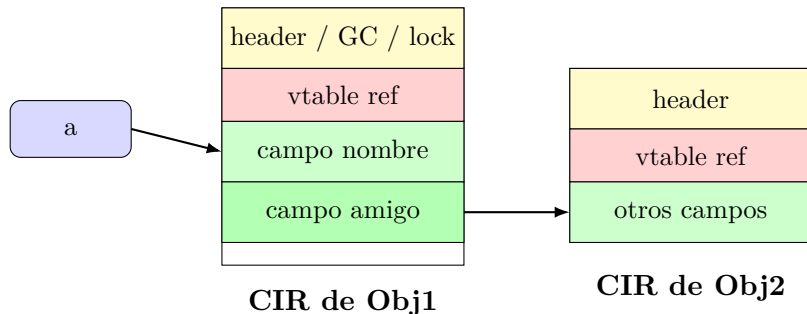


Resultado

Los objetos no marcados se consideran basura. El espacio que ocupaban queda libre para nuevas asignaciones.

- Mark & Sweep detecta ciclos.
- Puede dejar **fragmentación** si no compacta.

Integración directa con CIR



- El GC sigue **campos referencia** dentro del CIR.
- La **vtable** participa en el despacho dinámico, pero no decide si el objeto está vivo.
- Para el GC, lo importante es: **¿hay una referencia hacia este CIR?**

Contador de referencias

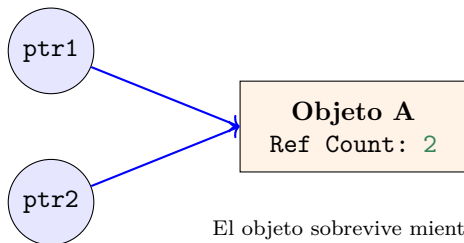
¿Qué es el Reference Counting?

Es una técnica de gestión de memoria donde cada objeto mantiene un ****contador interno**** de cuántas referencias apuntan a él.

La Regla Fundamental

- **Incremento:** Cuando se crea una nueva referencia al objeto.
- **Decremento:** Cuando una referencia sale de ámbito o se reasigna.
- **Liberación:** Si el contador llega a **0**, el objeto se destruye inmediatamente.

Visualización del Contador



El objeto sobrevive mientras exista al menos un dueño.

Trade-offs: Determinismo vs. Ciclos

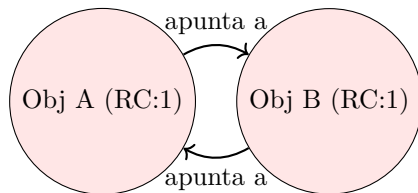
Ventajas

- **Determinismo:** La memoria se libera apenas deja de usarse (latencia baja).
- **Incremental:** No hay pausas largas de "Stop-the-world".
- **Simplicidad:** Fácil de implementar (ej. `std::shared_ptr` en C++).

Desventajas

- **Overhead:** Cada asignación requiere una operación atómica de suma/resta.
- **Ciclos de Referencia:** Si A apunta a B y B a A, el contador nunca llegará a 0.

El talón de Aquiles: Referencias Circulares



Memory Leak: Ninguna variable externa apunta a ellos, pero se mantienen vivos entre sí.

*Solución común: Uso de **Weak Pointers** (referencias débiles que no aumentan el contador).*

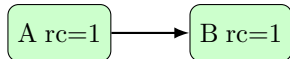
¿Dónde se utiliza?

- **C++:** `std::shared_ptr` y `std::weak_ptr`.
- **Objective-C / Swift:** ARC (Automatic Reference Counting).
- **Python / PHP:** Utilizan conteo de referencias como mecanismo principal, complementado con un recolector de ciclos.
- **Sistemas de Archivos:** Los *hard links* en Linux/Unix funcionan bajo este principio.

Comparación entre GC y Reference Counting

Reference Counting

Cada objeto mantiene un contador. Si el contador llega a 0, se libera.



- simple,
- liberación inmediata,
- no necesita recorrido global.

Problema de ciclos

Dos objetos pueden quedar apuntándose entre sí aunque ya no exista ninguna raíz.



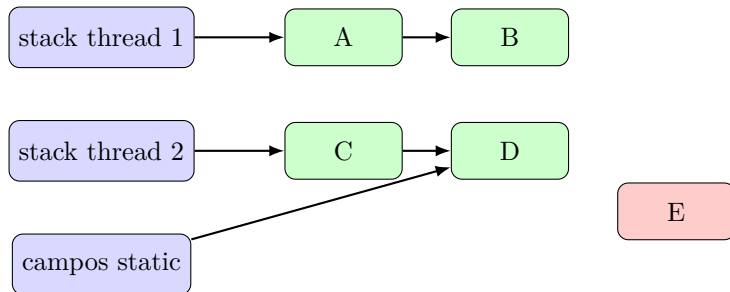
sin raíces

- el ciclo no baja a 0,
- esos objetos siguen ocupando memoria,
- un GC por alcanzabilidad sí puede liberarlos.

Ejemplo simple en Java

```
class Nodo {  
    Nodo sig;  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Nodo a = new Nodo();  
        Nodo b = new Nodo();  
        a.sig = b;  
        b.sig = a;    // ciclo  
  
        a = null;  
        b = null;  
    }  
}
```

Múltiples threads como raíces



- Cada thread aporta sus propias raíces desde su stack.
- El GC debe considerar **todos los hilos vivos**.
- Un objeto puede seguir vivo aunque ya no sea alcanzable desde un hilo, si todavía lo es desde otro.

¿Cómo lo hace Java?

- Java usa GCs modernos basados en **alcanzabilidad**, no solo en conteo de referencias.
- El runtime identifica raíces en:
 - stacks de threads,
 - referencias estáticas,
 - referencias internas de la VM.
- A partir de esas raíces recorre el heap y descubre el grafo vivo.
- Según el collector, luego barre, compacta o mueve objetos.

Coordinación con el sistema

El lenguaje administra **objetos** y el SO administra **memoria del proceso**. El GC puede liberar objetos sin devolver inmediatamente páginas al sistema operativo.

Idea final

Para entender GC en Java hay que unir: **raíces + heap + CIR + referencias + algoritmo de recorrido**.

Otros algoritmos de GC

- Reference Counting
- Mark & Sweep
- Mark & Compact
- Copying Collector
- Generational GC

Idea

Todos buscan balancear:

- performance
- uso de memoria
- pausas de ejecución

Relación con CIR

- Cada objeto en heap tiene un CIR
- El GC recorre objetos a través de referencias
- La vtable NO determina vida del objeto

Clave

El GC trabaja sobre referencias, no sobre tipos

Ejemplo en Java

```
Animal a = new Animal();  
a = null;
```

- El objeto queda sin referencias
- Es candidato a recolección

Coordinación con el Sistema Operativo

- El lenguaje gestiona el heap lógico
- El SO provee memoria física
- El GC no libera memoria al SO inmediatamente

Idea

El GC trabaja a nivel de objetos, el SO a nivel de páginas

Referencias

-  FISCHER, CHARLES N., CYTRON, RONALD K., LEBLANC, RICHARD J., 2009, *Crafting a Compiler (Chapter 11: Code Generation)*, Pearson Education.
-  AHO, ALFRED, SETHI, RAVI, ULLMAN, JEFFREY D., LAM, MONICA S., 2008, *Compiladores. Principios, técnicas y herramientas*, Addison Wesley.
-  SEBESTA, ROBERT W., 2016, *Concepts of Programming Languages*, Pearson.